LDraw 3D Web Viewer & Editor

A web-based viewer and editor for LDraw models built with TypeScript, React, Three.js, and Monaco Editor. View, edit, and manipulate LEGO models in 3D using the industry-standard LDraw format.

Table of Contents

- [Overview](#overview)
- [Quick Start](#quick-start)
- [Project Architecture](#project-architecture)
- [Project Structure](#project-structure)
- [NPM Scripts Reference](#npm-scripts-reference)
- [Core Technologies](#core-technologies)
- [LDraw Format & Coordinate System](#ldraw-format--coordinate-system)
- [Data Models](#data-models)
- [Monaco Editor Features](#monaco-editor-features)
- [Three.js Integration](#threejs-integration)
- [State Management](#state-management)
- [Testing Framework](#testing-framework)
- [Production Deployment](#production-deployment)
- [Development Workflow](#development-workflow)
- [Troubleshooting](#troubleshooting)
- [Contributing](#contributing)

Overview

This application is a 3D model viewer and editor for the LDraw format - the standard for digital LEGO modeling. It provides:

- **Real-time 3D visualization** of LDraw models using WebGL
- **Live code editing** with syntax highlighting and error detection
- **Interactive manipulation** of 3D objects with transform controls
- **Multi-selection capabilities** for batch operations
- **File management** with save/load functionality
- **Parts library integration** for resolving LDraw part references

Target Audience

This documentation is for developers who are:
- New to LDraw format and LEGO digital modeling
- Familiar with JavaScript/TypeScript but new to 3D web development
- Interested in understanding modern React + Three.js architecture
- Looking to contribute to or extend this codebase

Quick Start

Prerequisites

- **Node.js** (version 18.0 or higher)
- **npm** (comes with Node.js)
- **Modern web browser** (Chrome, Firefox, Safari, Edge)
- **Git** for version control

Installation

```bash
# Clone the repository
git clone https://github.com/your-repo/LDraw3DWebVis.git
cd LDraw3DWebVis

# Install dependencies
npm install

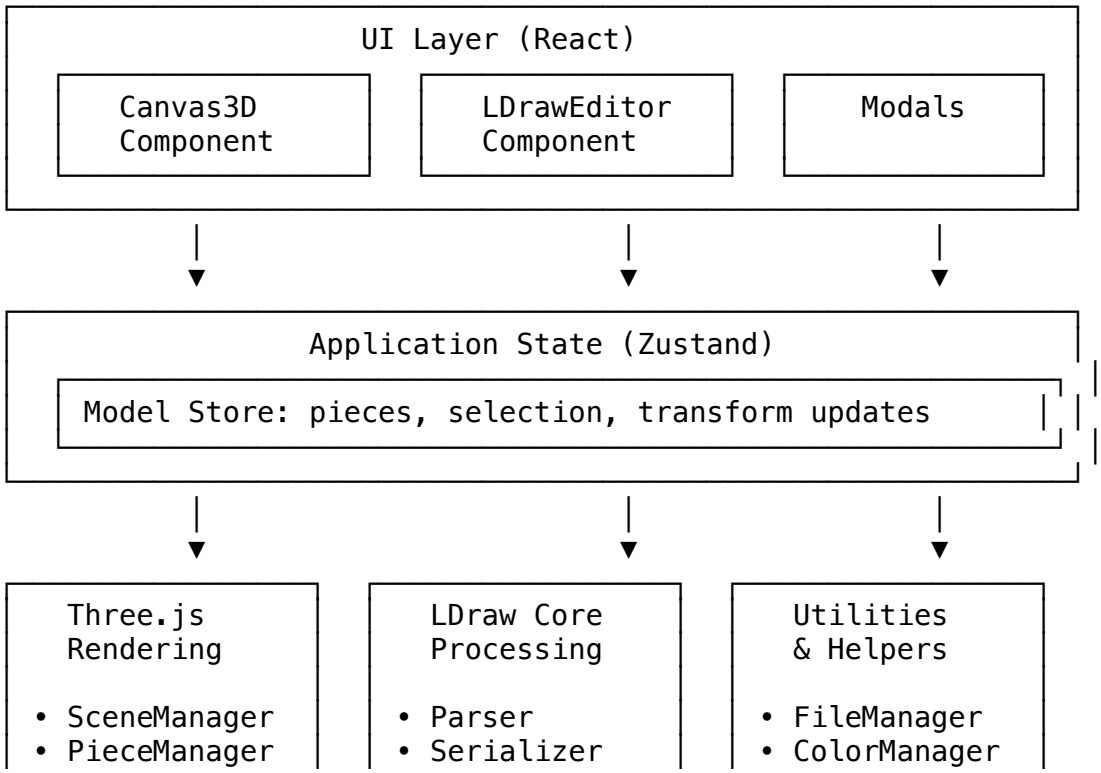
# Start development server
npm run dev
```

The application will open automatically at `http://localhost:5173`

Project Architecture

Architectural Overview

This application follows a **modular, layered architecture**:

```


```

```
| • Selection      |   | • DataModels     |   | • Transforms     |
| • Transforms     |   | • PartsManager   |   |                  |
└──────────────────┘   └──────────────────┘   └──────────────────┘
```

Design Principles

1. **Separation of Concerns**: Each module has a single responsibility
2. **Dependency Injection**: Components receive dependencies rather than creating them
3. **Event-Driven**: Components communicate through events and state changes
4. **Immutable State**: State updates create new objects rather than mutating existing ones
5. **Type Safety**: Comprehensive TypeScript interfaces and type checking

---

Project Structure

```
LDraw3DWebVis/
├── 📁 src/                          # Source code directory
│   ├── 📁 components/               # React UI components
│   │   ├── Canvas3D.tsx            # Main 3D viewport component
│   │   ├── LDrawEditor.tsx          # Monaco code editor component
│   │   ├── AppHeader.tsx           # Application header
│   │   ├── ResizablePaneContainer.tsx # Split-pane layout
│   │   └── 📁 modals/               # Modal dialog components
│   │       ├── ConfirmationModal.tsx # Generic confirmation dialog
│   │       └── FileNameModal.tsx    # File naming dialog
│   │
│   ├── 📁 ldraw/                    # LDraw format processing (Pure TypeScript)
│   │   ├── LDrawDataModels.ts      # TypeScript interfaces for LDraw data
│   │   ├── LDrawParser.ts          # Parses LDraw text into data models
│   │   ├── LDrawSerializer.ts      # Converts data models back to LDraw text
│   │   ├── ColorManager.ts         # Handles LDraw color definitions
│   │   └── PartsFileManager.ts     # Manages loading of LDraw part files
│   │
│   ├── 📁 three/                    # Three.js rendering and 3D logic
│   │   ├── SceneManager.ts         # Core Three.js scene management
│   │   ├── PieceManager.ts         # Manages 3D piece instances
│   │   ├── SelectionManager.ts     # Handles object selection in 3D
```

```
│   │   ├── SelectionGroupManager.ts # Multi-selection functionality
│   │   ├── TransformControlsManager.ts # 3D transform controls
│   │   ├── InputEventHandler.ts    # Mouse/keyboard input
processing
│   │   ├── LDrawLoaderModule.ts    # Three.js LDraw format loader
│   │   ├── LDrawTransforms.ts      # Coordinate system conversions
│   │   ├── PartGeometryLoader.ts   # Loads 3D geometry for LDraw
parts
│   │   ├── reconcileScene.ts       # Efficient scene updates
│   │   └── utils.ts               # Three.js utility functions
│   │
│   ├── 📁 state/                   # Application state management
│   │   └── useModelStore.ts        # Zustand store for app state
│   │
│   ├── 📁 utils/                   # General utility modules
│   │   └── SceneFileManager.ts     # File I/O operations
│   │
│   ├── 📁 types/                   # TypeScript type definitions
│   │   └── three-ext.d.ts         # Three.js type extensions
│   │
│   ├── 📁 test/                    # Integration and unit tests
│   │   ├── setup.ts               # Test environment configuration
│   │   └── *.test.ts              # Various test suites
│   │
│   ├── App.tsx                     # Root React component
│   ├── App.css                     # Application styles
│   ├── main.tsx                    # Application entry point
│   └── index.css                   # Global styles
│
├── 📁 public/                      # Static assets served directly
│   └── 📁 scenes/                  # Sample LDraw model files
│       ├── default.ldr             # Default demo model
│       └── *.ldr                   # Other example models
│
├── 📁 ldraw/                       # LDraw parts library (478MB)
│   ├── parts/                      # Individual LEGO part
definitions
│   ├── p/                          # Primitive geometric shapes
│   ├── LDConfig.ldr               # Color definitions
│   └── ...                        # Complete LDraw library
structure
│
├── 📁 docs/                        # Project documentation
│   ├── DeployingPartsApi.md        # Production deployment guide
│   ├── ldraw_specs.MD              # LDraw format specification
│   └── MPD.MD                      # Multi-Part Document format
info
│
├── 📄 package.json                 # Node.js project configuration
├── 📄 vite.config.ts              # Vite build tool configuration
```

```
├── 📄 vitest.config.ts           # Vitest testing configuration
├── 📄 tsconfig.json              # TypeScript configuration
├── 📄 eslint.config.js           # ESLint code quality
configuration
└── 📄 Readme.MD                   # This file
```

Key Directory Purposes

`/src/components/` - React UI Layer
Contains all React components that make up the user interface. Each component handles rendering specific UI elements and user interactions.

`/src/ldraw/` - LDraw Processing Engine
Pure TypeScript modules (no React/DOM dependencies) that handle:
- Parsing LDraw text files into structured data
- Converting data back to LDraw format
- Managing color definitions and part libraries
- Type-safe data models for all LDraw constructs

`/src/three/` - 3D Rendering Engine
Three.js-specific code that manages:
- 3D scene creation and management
- Object selection and manipulation
- Transform controls for moving/rotating objects
- Efficient scene updates and rendering optimization
- Coordinate system conversions between LDraw and Three.js

`/src/state/` - Application State
Zustand-based state management providing:
- Centralized model data storage
- Selection state management
- Reactive updates to UI components
- Immutable state update patterns

NPM Scripts Reference

Development Scripts

`npm run dev`
Starts the development server with hot reload on port 5173.

`npm run build`
Creates optimized production build (excludes 478MB LDraw parts library).

`npm run build:full`
Full build with TypeScript compilation check.

`npm run preview`

Preview production build locally on port 4173.

Code Quality Scripts

`npm run lint`
Runs ESLint to check code quality.

Testing Scripts

`npm test`
Runs all tests once.

`npm run test:watch`
Runs tests in watch mode for development.

Phase-Specific Tests
```bash
npm run test:phase2  # Core rendering tests
npm run test:phase3  # Scene reconciliation tests
npm run test:phase4  # Selection and transform tests
npm run test:phase5  # Multi-selection tests
```

Core Technologies

React 17 with TypeScript
- **Purpose**: UI framework for component-based architecture
- **Key Features**: Hooks, functional components, strict typing

Three.js 0.175.0
- **Purpose**: 3D graphics rendering engine
- **Key Features**: WebGL abstraction, scene graph, built-in LDraw loader

Monaco Editor
- **Purpose**: Advanced code editor (same engine as VS Code)
- **Key Features**: Syntax highlighting, error detection

Zustand 4.4.1
- **Purpose**: Lightweight state management
- **Key Features**: Simple API, TypeScript support, minimal boilerplate

Vite 6.2.0
- **Purpose**: Modern build tool and development server
- **Key Features**: Fast HMR, ES modules, optimized builds

Vitest 3.1.1
- **Purpose**: Unit testing framework
- **Key Features**: Vite integration, fast execution, Jest compatibility

---

LDraw Format & Coordinate System

LDraw Format Overview

LDraw uses plain text files with specific syntax:

```ldraw
0 Name: model.ldr
0 Author: Your Name

1 4 0 0 0 1 0 0 0 1 0 0 0 1 3001.dat
1 15 0 -8 0 1 0 0 0 1 0 0 0 1 3001.dat
```

Line Type Breakdown

- **Type 0**: Metadata and comments
- **Type 1**: Part/submodel references
  ```
  1 [color] [x] [y] [z] [a] [b] [c] [d] [e] [f] [g] [h] [i]
[filename]
  ```

Coordinate System

LDraw uses a **right-handed coordinate system** with:
- **X-axis**: Positive points right
- **Y-axis**: **Negative points up** (unusual)
- **Z-axis**: Positive points toward viewer

Coordinate System Conversion

```typescript
// LDraw: -Y is up, Three.js: +Y is up
function ldrawToThreePosition(ldrawPos: Vector3): Vector3 {
  return new Vector3(ldrawPos.x, -ldrawPos.y, ldrawPos.z);
}
```

LDraw Units
- **1 LDraw unit = 0.4mm** in real world
- Standard brick height: 9.6 LDraw units
- Standard stud spacing: 20 LDraw units

Data Models

LDrawPiece Interface
```typescript
interface LDrawPiece {
  id: string;                    // Unique identifier
```

```
  partId: string;               // Part filename (e.g., "3001.dat")
  position: Vector3;            // 3D coordinates in LDraw space
  rotationMatrix: RotationMatrix; // 3×3 rotation as 9-element array
  colorCode: number;            // LDraw color code
}
```

LDrawModel Interface
```typescript
interface LDrawModel {
  pieces: LDrawPiece[];         // Array of all pieces in model
  name?: string;                // Model name from metadata
  author?: string;              // Author from metadata
  description?: string;         // Description from metadata
}
```

Monaco Editor Features

File Management

- **New**: Creates empty LDraw template with metadata
- **Load**: Scene selector dropdown for available models
- **Save**: Saves to `/public/scenes/` directory

Live Editing

- **Real-time sync**: Changes reflected in 3D view after debounce
- **Error handling**: Parse errors displayed in UI
- **Smart updates**: Only updates changed pieces

LDraw Syntax Reference

```
1 [color] [x] [y] [z] [a] [b] [c] [d] [e] [f] [g] [h] [i] [partfile]
```

Example:
```ldraw
1 4 0 0 0 1 0 0 0 1 0 0 0 1 3001.dat
```

Common Color Codes
- `0` = Black, `4` = Red, `14` = Yellow, `15` = White
- `2` = Green, `1` = Blue, `6` = Brown, `8` = Dark Gray

Production Deployment

Quick Start (Single Server)

The simplest deployment runs both the app and parts library on the same server:

```bash
npm run build       # Build the application
npm start           # Start production server on port 3000
```

Open http://localhost:3000 to view the application.

Production Server (`server.cjs`)

The included Express server serves:
- Built application from `dist/`
- LDraw parts library from `ldraw/`
- Scene files with save/load functionality

Environment Variables:
Variable	Default	Description
`PORT`	3000	Server port
`LDRAW_DIR`	`./ldraw`	Path to LDraw parts library

```bash
PORT=8080 npm start                    # Custom port
LDRAW_DIR=/path/to/ldraw npm start     # Custom library path
```

Separate Parts API (Advanced)

For larger deployments, host the parts library separately:

1. Deploy Parts API as separate service (see `docs/DeployingPartsApi.md`)
2. Set `VITE_LDRAW_API_URL` before building:

```bash
VITE_LDRAW_API_URL=https://parts-api.yourdomain.com npm run build
```

Build Commands

```bash
npm run build       # Create production build (excludes ldraw/)
npm run build:full  # Build with TypeScript check
npm run preview     # Preview build locally (port 4173)
npm start           # Run production server (port 3000)
```

See `docs/DeployingPartsApi.md` for full deployment options.

Development Workflow

Getting Started

```bash
git clone https://github.com/yourusername/LDraw3DWebVis.git
cd LDraw3DWebVis
npm install
npm run dev
```

Running Tests

```bash
npm run test:watch  # Watch mode for TDD
npm test           # Run once for CI
```

Troubleshooting

LDraw Parts Not Loading

```bash
# Check ldraw directory exists
ls ldraw/

# Test parts API
curl http://localhost:5173/api/parts/3001.dat
```

Monaco Editor Not Loading

- Check `@monaco-editor/react` installation
- Clear browser cache

Three.js Scene Not Rendering

- Check WebGL support in browser
- Requires: Chrome 57+, Firefox 52+, Safari 11+

Contributing

Code Standards

- Use strict TypeScript type checking
- Use functional React components with hooks
- Always dispose Three.js geometries and materials
- Include tests for new functionality

Pull Request Process

1. Create feature branch
2. Make changes with tests
3. Run `npm test && npm run lint`
4. Create pull request with descriptive title

For the most up-to-date information, refer to the source code and inline documentation.